ELEC6234
Embedded Processor Design

FPGA synthesis of a microprocessor core
Notes on ALU synthesis II
Branching

UNIVERSITY OF Southampton
School of Electronics and Computer Science

Carry logic can be implemented by a simple extension to the ALU

A	*B*	*Cin*	*Sum*	*Cout*
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

When coding addition as A+B,
we don't know Cin at MSB, but we know Sum[7]

Carry is set at MSB slice if:

A[7] & B[7] | B[7] & ~Sum[7] | A[7] & ~Sum[7]

In subtraction A-B, complement B[7]

Inverted carry in subtraction

A	*~B*	*Cin*	*Sum*	*~Cout*
0	*0*	*0*	*0*	*1*
0	*0*	*1*	*1*	*1*
0	*1*	*0*	*1*	*1*
0	*1*	*1*	*0*	*0*
1	*0*	*0*	*1*	*1*
1	*0*	*1*	*0*	*0*
1	*1*	*0*	*0*	*0*
1	*1*	*1*	*1*	*0*

In subtraction A-B = A + ~B + 1 Carry out at the MSB is inverted to represent Borrow and is:

~A[7] & B[7] | ~A[7] & Sum[7] | B[7] & Sum[7]

Detection of 2's complement overflow

Overflow occurs in addition if the signs of both operands are the same, but the result sign is different, i.e.

A[7] & B[7] & ~Sum[7] | ~A[7] & ~B[7] & Sum[7]

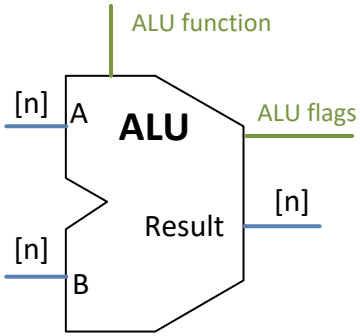
In subtraction A-B, complement B[7]

A	*B*	*Cin*	*Sum*	*Cout*
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

ALU with condition codes (MIPS style)

```
//-----------------------------------------------------
// File Name   : alu.sv
// Function    : ALU module for picoMIPS
// Version: 1,  only 8 funcs
// Author:  tjk
// Last rev. 17 Feb 21
//-----------------------------------------------------

`include "alucodes.sv"
module alu #(parameter n =8) (
  input logic [n-1:0] a, b, // ALU operands
  input logic [2:0] func, // ALU function code
  output logic [3:0] flags, // ALU flags V,N,Z,C
  output logic [n-1:0] result // ALU result
);
…
```

```
//-----------------------------------------------------
// File Name   : alucodes.sv
// Function    : pMIPS ALU function code definitions
// Author:   tjk
// Last rev. 23 Oct 12
//-----------------------------------------------------
//
`define RA   3'b000
`define RB   3'b001
`define RADD  3'b010
`define RSUB  3'b011
`define RAND  3'b100
`define ROR   3'b101
`define RXOR  3'b110
`define RNOR  3'b111
```



Extended ALU - cont

alu.sv

```
…
//------------- code starts here ---------

// create an n-bit adder
// and then build the ALU around the adder
logic[n-1:0] ar,b1; // temp signals
always_comb
begin
  if(func==`RSUB)
    b1 = ~b + 1'b1; // 2's complement subtrahend
  else b1 = b;

  ar = a+b1; // n-bit adder
end // always_comb

…
```

This always_comb block creates an adder and logic for –B needed in subtraction.

The adder result (ar) will be used in the always_comb block that follows.

alu.sv

Extended ALU - cont

```
...
// create the ALU, use signal ar in arithmetic operations
always_comb
begin
  //default output values; prevent latches
  flags = 3'b0;
  result = a; // default
  case(func)
    `RA   : result = a;
    `RB   : result = b;
    `RADD  : begin
          result = ar; // arithmetic addition
          flags[3] = a[7] & b[7] & ~result[7] |  ~a[7] & ~b[7]    &  result[7]; //V
         flags[0] = a[7] & b[7]  |  a[7] & ~result[7] | b[7]    & ~result[7]; // C
         end
   `RSUB  : begin
        result = ar; // arithmetic subtraction
         flags[3] = a[7] & ~b[7] & ~result[7] |  ~a[7] & b[7]  & result[7]; // V
        // C - note: picoMIPS inverts carry when subtracting
        flags[0] = ~a[7] & b[7] |  ~a[7] & result[7] | b[7] & result[7];
       end
   `RAND  : result = a & b;
   `ROR   : result= a | b;
   `RXOR  : result = a ^ b;
   `RNOR  : result = ~ (a | b);
  endcase
...
```

This always_comb block creates the ALU.

Note the logic for ALU flags:
V – overflow
C - carry on MSB

See following slides for explanation of V and C logic

alu.sv

Extended ALU - cont

```
…

  // calculate flags Z and N
  flags[1] = result == {n{1'b0}}; // Z
  flags[2] = result[n-1]; // N

 end //always_comb

endmodule //end of module ALU
```

The logic for ALU flags:
Z – zero result
N - negative result

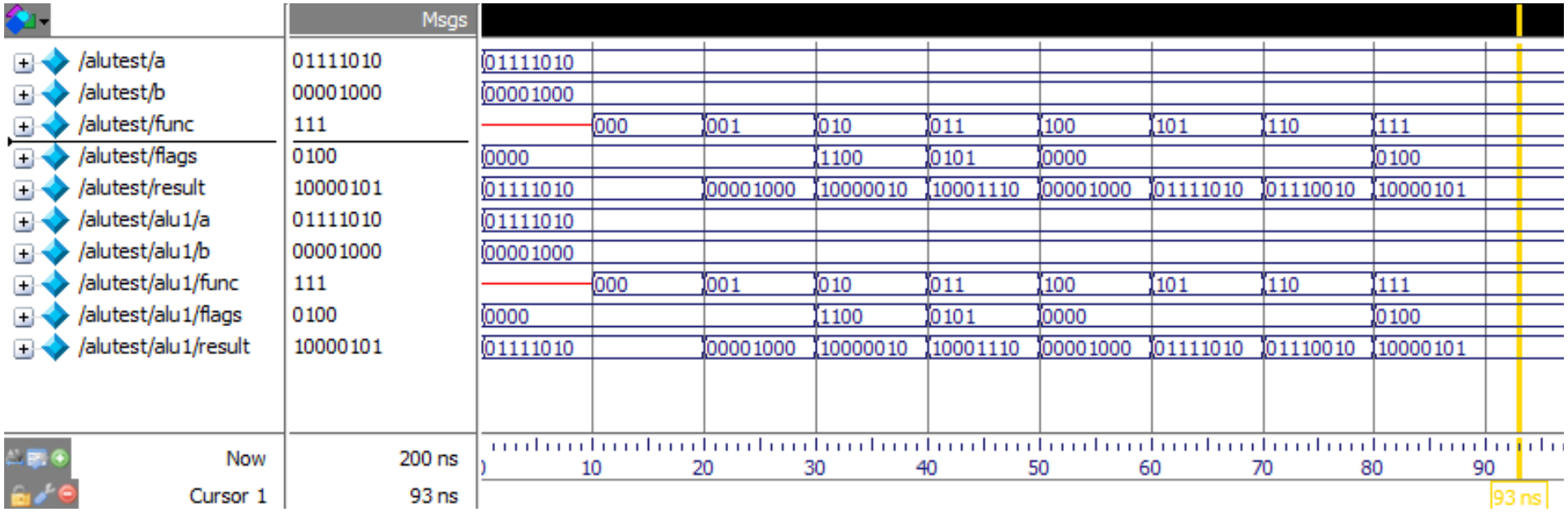
In this approach the branch logic can be implemented using the ALU condition code, a RISC-V style branch block is not necessary

ALU testbench

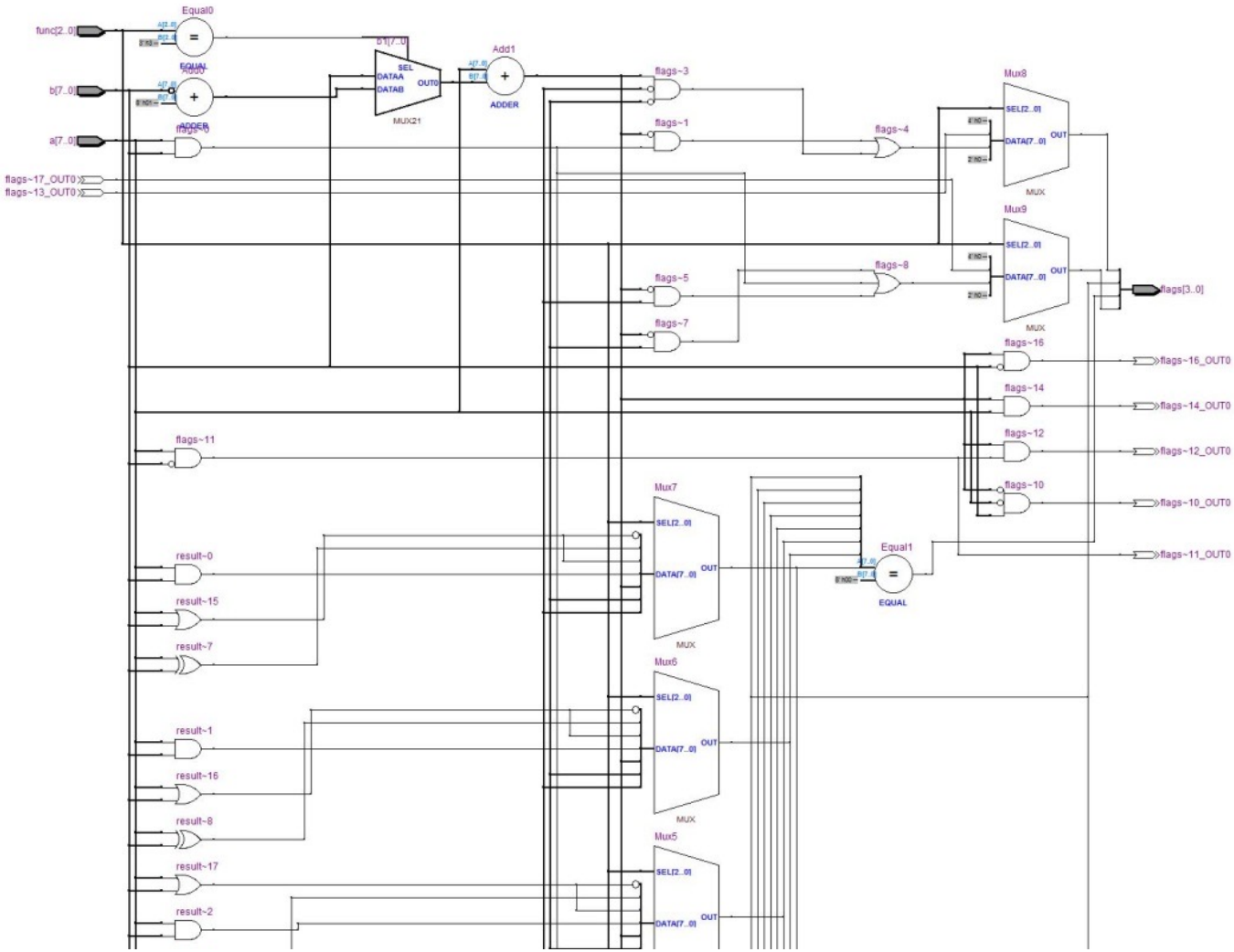
```
//-----------------------------------------------------
// File Name   : alutest.sv
// Function    : Testbench for picoMIPS ALU
// Version: 1,  only 8 ALU funcs
// Author:  tjk
// Last rev. 23 Oct 12
//-----------------------------------------------------
`include "alucodes.sv"
module alutest;
parameter n =8; // data bus width
logic [n-1:0] a, b; // ALU input operands
logic [2:0] func; // ALU function code
logic [3:0] flags; // ALU flags V,N,Z,C
logic [n-1:0] result; // ALU result

alu #(.n(n)) alu1 (.*);  // create alu object

…
```

```
…
//------------- code starts here ---------
initial
  begin  a= 8'h7a;  b= 8'h08;
  // test all functions
  #10 func = `RA; // result = a
  #10 func = `RB; // result = b
  #10 func = `RADD; // result = a+b
  #10 func = `RSUB; // result = a-b
  #10 func = `RAND; // result = a & b
  #10 func = `ROR; // result = a | b
  #10 func = `RXOR; // result = a ^ b
  #10 func = `RNOR; // result = ~ (a | b)
  end //initial

endmodule //end of alutest
```

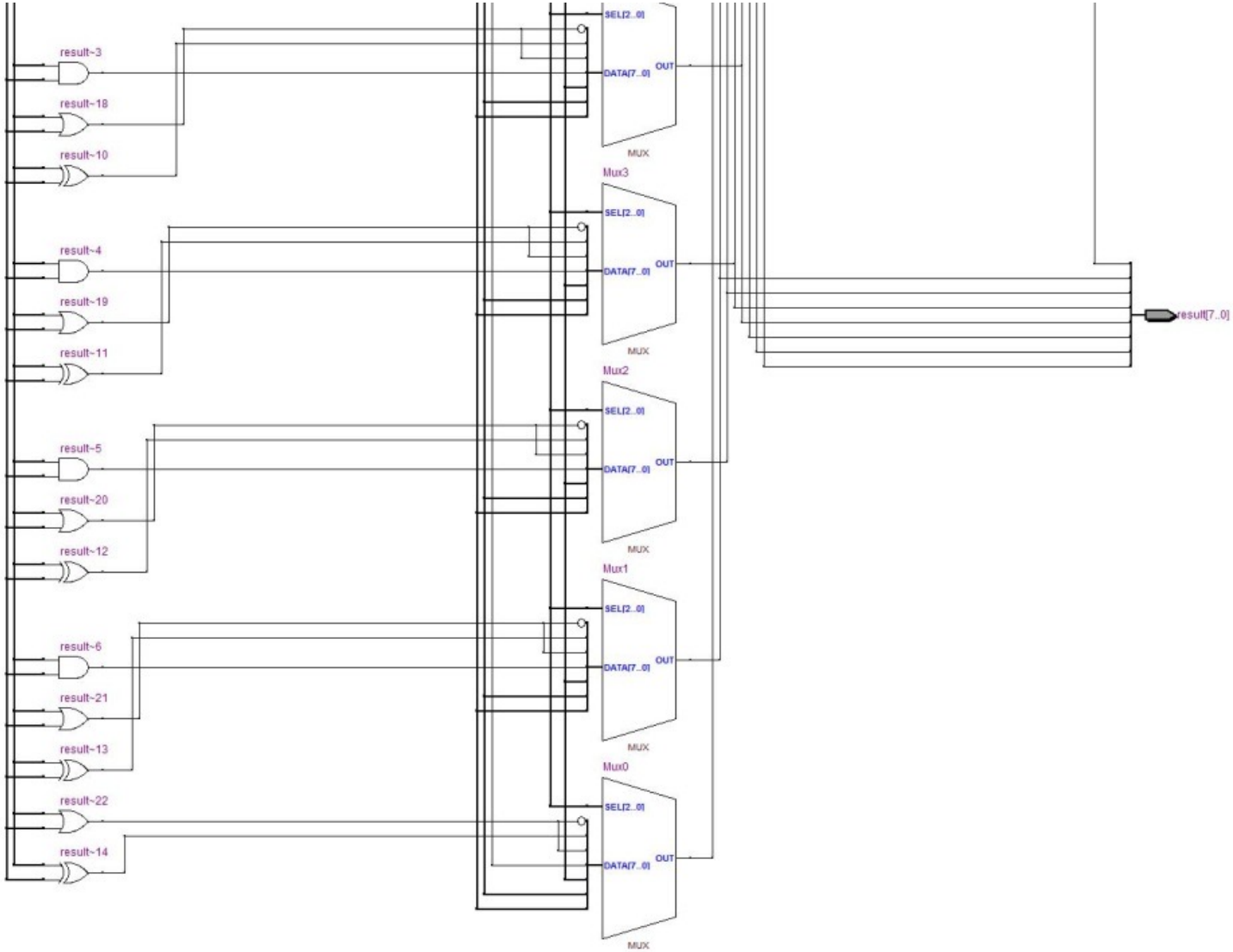
Modelsim waveforms



Altera Quartus synthesis



Altera Quartus synthesis - cont



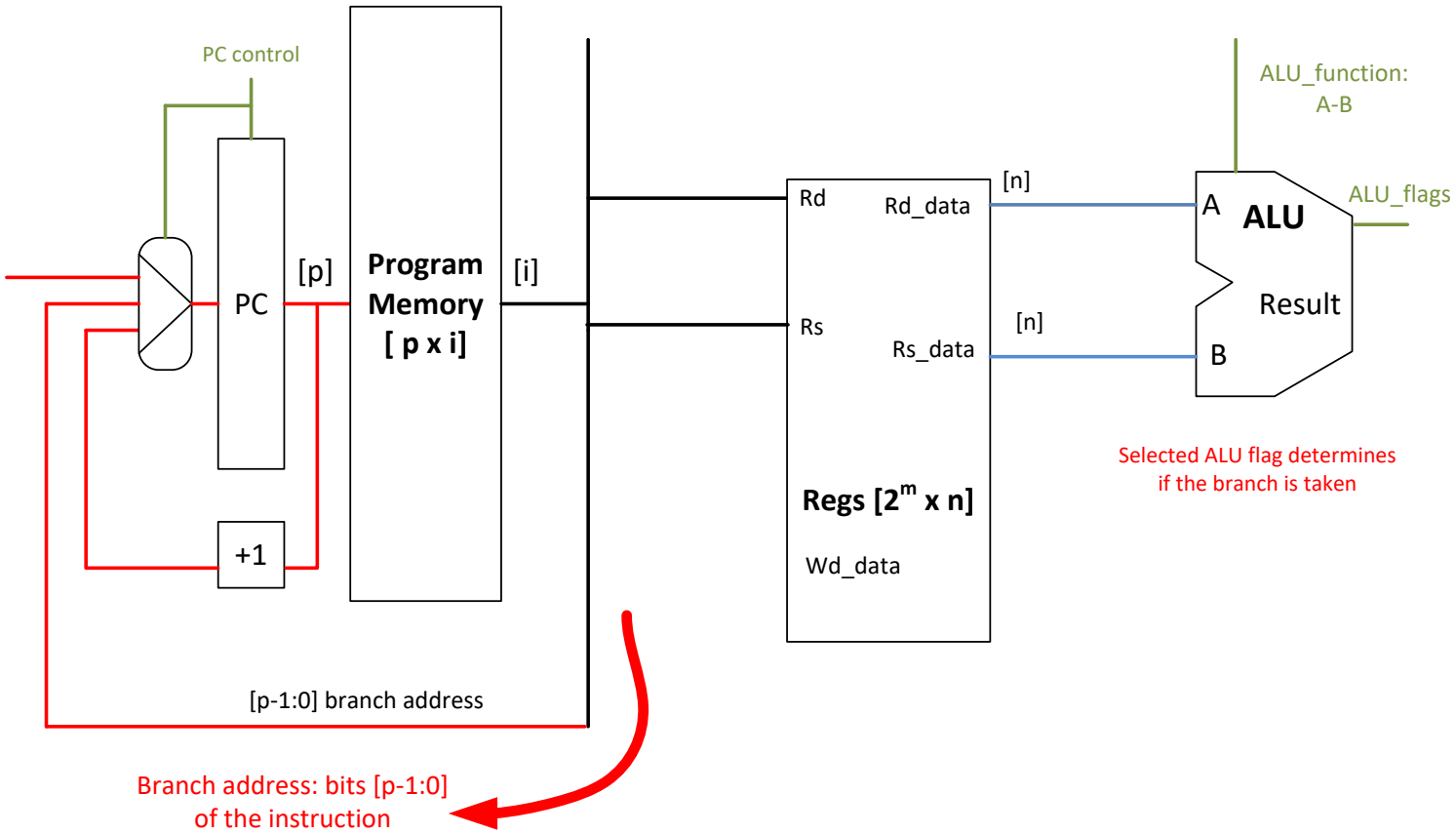
Conditional branches absolute or relative

Absolute conditional branch:
PC <= (condition? branch address: PC+1)
where branch address is: Instruction[p-1:0]

Relative conditional branch:
PC <= (condition? PC + Instruction[p-1:0]: PC+1)

ALU is programmed for subtraction: Rd-Rs

Subtraction result is not stored but ALU flags: Z,N,C are used to determine if the branch should be taken



Summary

- We discussed two approaches to ALU synthesis – MIPS style and RISC-V style
- Normally RISC-V style is recommended
- Next we will talk about registers and RAM memory